TECNOLÓGICO DE MONTERREY

Escuela de Ingeniería y Ciencias · Departamento de Ciencias Computacionales

PROYECTO INTEGRADOR · CORRIDA V5

Formulación corregida y comparación de algoritmos — bitácora razonada paso a paso (nivel 1)

REINFORCE, PPO, SAC, DDPG y árbol de decisión para proponer el máximo con pocas mediciones

Continuación de la V4 con tres correcciones de raíz al objetivo de entrenamiento. Incluye el razonamiento de cada paso, los resultados con desviaciones entre semillas, el efecto del ruido de medición, la revisión de los casos reales, las limitaciones y los siguientes pasos.

Sponsor: Prof. Gerardo Jesús Camacho González, PhD. · Monterrey, Nuevo León, México · Octubre de 2026

Índice

1. Resumen ejecutivo 3

2. Paso a paso de la corrida V5 5

Paso 1 — Las tres correcciones de raíz y su implementación en REINFORCE 6

Paso 2 — Malla de REINFORCE corregido (N = 7, K = 5) 7

Paso 3 — ¿Dónde hay margen? Barrido de N y K, con un control más fuerte 8

Paso 4 — Cinco algoritmos sobre la misma formulación 9

Paso 5 — Ruido de medición (1 y 2 W) 11

Paso 6 — Revisión de los casos reales (¿por qué el árbol falla en algunos?) 12

3. Conclusiones 14

4. Siguientes pasos y preguntas para el profesor 15

5. Limitaciones y supuestos declarados 16

6. Mapa de archivos y cómo reproducir 17

Anexo. Código clave 18

1. Resumen ejecutivo

La V4 mostró que, con la formulación inicial, REINFORCE-MLP y PPO no superaban a una referencia sin aprendizaje con N = 7 puntos de barrido y K = 5 muestras. La causa probable era el objetivo: se premiaba la potencia *promedio* de las muestras, lo que lleva a σ → 0 y no premia al *mejor* punto, que es el que se elige. Esta corrida (V5) corrige tres cosas de raíz (recompensa de grupo, σ fija o decreciente, más muestras al entrenar que al evaluar) y compara **REINFORCE, PPO, SAC, DDPG y un árbol de decisión recalculado en cada iteración** sobre la misma formulación, siempre sin GMPP en entrenamiento, con el rango fijado por la Isc nominal y en el nivel 1 (potencia estática).

Respuesta corta. Con las correcciones, el enfoque propuesto por el profesor **sí supera a los controles sin aprendizaje con pocas mediciones**: con N = 5 (10 mediciones) DDPG y el árbol ganan ≈ 3 pp de exactitud final y de energía; con N = 7 (12 mediciones) el árbol alcanza 98.5 % / 96.5 % frente a 97.2 % / 95.4 % del control, y supera incluso al control con 15 mediciones. La ventaja se mantiene con ruido de medición de 1 y 2 W. En los 4 casos reales los resultados son mixtos y el caso 3 (Isc baja) es débil para todos los métodos. No supera la exactitud del MSAPSO del paper (≈ 100 % con 175 mediciones): la ganancia está en el costo en mediciones.

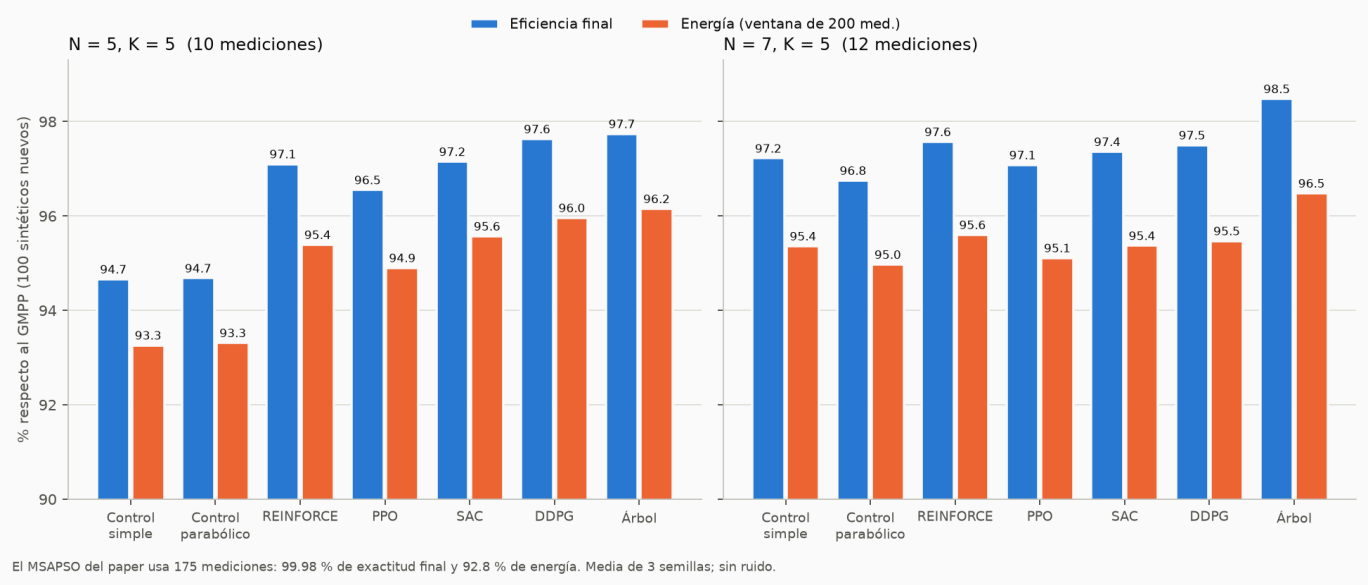


Figura A. Comparación en 100 patrones sintéticos nuevos (K = 5; media de 3 semillas; sin ruido). Eficiencia final = potencia en el punto elegido / GMPP; energía = promedio en una ventana de 200 mediciones. El MSAPSO del paper usa 175 mediciones.

Hallazgos principales

1. **Las correcciones sirven, pero donde hay margen:** con N = 7 mueven poco el resultado de REINFORCE (+0.3 pp, dentro del ruido); con N de 3 a 5 ganan ≈ 2–3 pp sobre los controles

(Pasos 2 y 3). Un control sin aprendizaje más fuerte (interpolación parabólica) no mejora al simple.

2. **Mejores algoritmos:** el árbol, DDPG y SAC lideran; PPO y REINFORCE quedan algo por debajo, aunque por encima de los controles con N de 3 a 5 (Paso 4). El árbol entrena en 2 segundos, contra 160–260 s de SAC y DDPG.
3. **Frontera de mediciones:** con 8 mediciones el RL logra la energía que el control simple logra con ≈ 10–12 (Figura B).
4. **Ruido de 1 y 2 W:** la ventaja persiste; el árbol es el mejor con N = 7 y DDPG con N = 5 (Paso 5).
5. **Casos reales:** solo son 4. El caso 3 (Isc = 3.25 A) es débil para todos porque el rango se fija con la Isc nominal y casi no hay puntos útiles en el barrido; entrenar con atenuación global mejora en parte (Paso 6; esa idea se propuso después de ver el fallo).
6. **Corrección a la V4:** la «cota supervisada» no era un techo; el árbol, con solo potencias medidas, la supera.

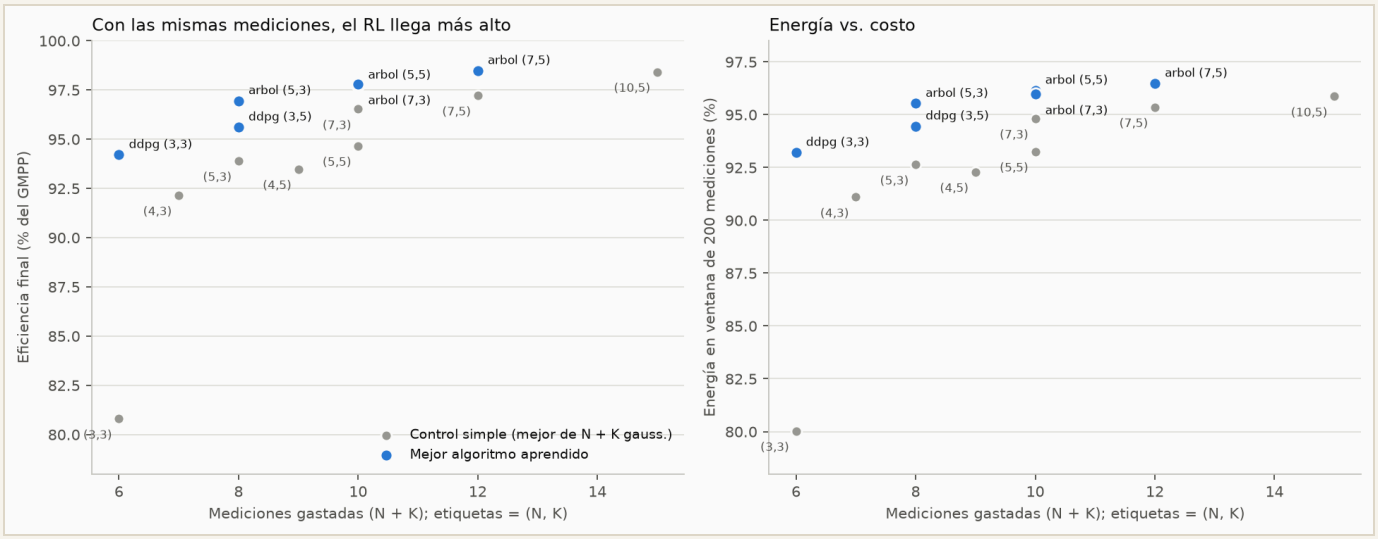


Figura B. Costo (mediciones N + K) contra exactitud final y energía en 100 sintéticos nuevos, sin ruido: el control simple (gris, etiquetas = (N, K)) contra el mejor algoritmo aprendido en cada combinación.

2. Paso a paso de la corrida V5

Cada paso indica **qué se hizo**, **por qué** (el razonamiento), cómo se **verificó**, el **resultado** y los **archivos** que lo respaldan.

Documento vivo: cada entrada registra QUÉ se hizo, POR QUÉ (razonamiento), cómo se VERIFICÓ, QUÉ RESULTÓ y QUÉ ARCHIVOS lo respaldan. De aquí se compila el PDF.

Contexto y punto de partida (octubre 2026)

- La V4 (carpeta `Proyecto Integrador Paneles_V4/`) no se modifica. Implementó el esquema del profesor (N potencias medidas como observación, MLP, política gaussiana, muestreo de puntos aledaños) y concluyó que, con esa formulación, REINFORCE-MLP y PPO no superaban a la referencia sin aprendizaje «mejor de 7 + 5 gaussianas»; 15 veces más entrenamiento no cambiaba el resultado.
- Causa probable identificada en la V4: la recompensa promediaba la potencia de las muestras, lo que empuja σ a cero (en PPO σ cayó a 0.013) y no premia al MEJOR punto, que es el que se elige al final.
- Esta corrida (V5) aplica tres correcciones de raíz y, como el resultado mejora, compara REINFORCE, PPO, SAC, DDPG y un árbol de decisión sobre la MISMA formulación. Sigue en el NIVEL 1 (potencia estática, sin dinámica del convertidor), sin GMPP real en entrenamiento y con el rango de corrientes fijado con la Isc nominal (8.9 A).
- Evaluación común: 100 patrones sintéticos nuevos (semilla 2026, distintos de los de entrenamiento) y los 4 casos reales; eficiencia final (punto elegido / GMPP), energía en una ventana de 200 mediciones y número de mediciones (N + K).

PASO 1

Paso 1 — Las tres correcciones de raíz y su implementación en REINFORCE

Qué hice. `reinforce_nn_V5.py` : (a) recompensa de GRUPO: `max` = la mejor de las K muestras (potencia medida) o `gain` = max(0, mejor muestra − mejor punto del barrido), en lugar de la potencia media de cada muestra; el gradiente pasa a ser $\nabla J = E[(G - b)\cdot\Sigma_k (a_k - \mu)/\sigma^2\cdot\nabla\mu]$. (b) σ FIJA o DECRECIENTE (de σ_0 a σ_1), sin aprenderla. (c) K de entrenamiento distinto del K de evaluación (5 o 20 muestras al entrenar, 5 al evaluar). Baselines de valor (V(obs) aprendida) o de media móvil.

Por qué. Se ataca la causa probable de la V4: el objetivo promedio premia concentrar todas las muestras en el mismo punto; el objetivo de grupo premia que ALGUNA muestra supere lo ya conocido, que es lo que se usa al final.

Verificación. Solo potencias medidas en la recompensa y en la observación (sin GMPP); evaluación en patrones nuevos.

Archivos. `reinforce_nn_V5.py` .

PASO 2

Paso 2 — Malla de REINFORCE corregido (N = 7, K = 5)

Qué hice. `grid_reinforce_V5.py` : recompensa (`mean` de la V4, `max` , `gain`) × baseline (valor, media móvil) × K de entrenamiento (5, 20) × σ (0.05 fija; decreciente 0.15 → 0.03), 3 semillas, 4000 actualizaciones. 24 configuraciones.

Resultado. La mejor configuración en sintéticos fue `gain` , baseline de valor, K de entrenamiento 5, σ = 0.05 fija: 97.54 % de exactitud final y 95.60 % de energía (referencia sin aprendizaje: 97.22 y 95.36 %; la V4: 96.7 y 94.9 %). La diferencia con la referencia (≈ +0.3 pp) está dentro del ruido. El punto μ solo (una medición) sube de ≈ 94.4 a ≈ 95.7 %.

Lectura. Con N = 7 las correcciones por sí solas mueven poco el resultado: ya queda poco margen sobre un control simple. La pregunta pasó a ser DÓNDE hay margen (Paso 3).

Archivos. `grid_reinforce_V5.py` , `resultados_V5/reinforce_grid_V5.txt|json` .

PASO 3

Paso 3 — ¿Dónde hay margen? Barrido de N y K, con un control más fuerte

Qué hice. `margen_NK_V5.py` y `control_parabolico_V5.py`: para N ∈ {3, 4, 5, 7, 10} y K ∈ {3, 5}, comparé tres cosas sobre 100 sintéticos nuevos: el control simple («mejor de N + K gaussianas»), un control sin aprendizaje más fuerte (ajuste de una parábola a los 3 puntos vecinos del mejor del barrido y K gaussianas alrededor de su vértice) y REINFORCE corregido. También una cota supervisada de diagnóstico (MLP con el GMPP como etiqueta; NO es un método válido bajo la regla del profesor).

Por qué. El profesor pidió «muy pocos muestreos»; con N = 7 un control simple ya iguala, pero con N pequeño la información del barrido es escasa y un modelo podría aportar más. El control parabólico evita comparar contra un control demasiado débil.

Resultado (final / energía en sintéticos).

N, K (MEDICIONES)	CONTROL SIMPLE	CONTROL PARABÓLICO	REINFORCE CORREGIDO
3, 3 (6)	80.8 / 80.0	86.7 / 85.8	90.3 / 89.2
5, 3 (8)	93.9 / 92.6	94.0 / 92.7	96.4 / 94.9
5, 5 (10)	94.7 / 93.3	94.7 / 93.3	96.9 / 95.2
7, 5 (12)	97.2 / 95.4	96.8 / 95.0	97.5 / 95.6
10, 5 (15)	98.4 / 95.9	98.5 / 96.0	98.2 / 95.7

Lectura. El control parabólico NO mejora al simple; REINFORCE corregido gana ≈ 2–3 pp con N entre 3 y 5 y casi nada con N ≥ 7. El margen está en pocas mediciones.

Archivos. `margen_NK_V5.py`, `control_parabolico_V5.py`, `resultados_V5/margen_NK_V5.txt`, `resultados_V5/control_parabolico_V5.txt`.

PASO 4

Paso 4 — Cinco algoritmos sobre la misma formulación

Qué hice. `algoritmos_V5.py` y `comparar_algoritmos_V5.py`, con N ∈ {3, 5, 7}, 3 semillas, K = 3 y 5, σ de evaluación = 0.05·Isc_nominal: REINFORCE corregido (recompensa `gain`, baseline de valor, σ fija); PPO, SAC y DDPG de stable-baselines3 sobre un entorno de UN paso (`pvsim_V5/bandit_env_V5.py`, recompensa `gain`, banco precalculado de 10 000 curvas), PPO con σ congelada y DDPG con ruido gaussiano de exploración; y el ÁRBOL DE DECISIÓN dentro del bucle de RL, recalculado en cada iteración: se muestrean K = 10 acciones alrededor de la propuesta actual, se mide la potencia y el árbol (profundidad 10) se reajusta con la mejor acción MEDIDA de cada patrón si mejora al barrido, con σ decreciente (0.15 → 0.04) en 15 iteraciones. Solo potencias medidas. Entrenamiento: PPO 300 mil pasos (≈ 25 s), SAC y DDPG 100 mil pasos (160–260 s), árbol ≈ 2 s.

Evaluación. Se mide el barrido de N puntos, el modelo propone μ y se toman K muestras N(μ, σ²); se elige la mejor leída.

Resultado (K = 5, sin ruido; 100 sintéticos nuevos, final / energía; media de 3 semillas).

N (MEDICIONES)	CONTROL SIMPLE	CONTROL PARABÓLICO	REINFORCE	PPO	SAC	DDPG	ÁRBOL
3 (8)	83.2 / 82.2	88.4 / 87.4	93.9 / 92.6	93.9 / 92.6	95.6 / 94.4	95.6 / 94.5	95.5 / 94.3
5 (10)	94.7 / 93.3	94.7 / 93.3	97.1 / 95.4	96.6 / 94.9	97.2 / 95.6	97.6 / 96.0	97.7 / 96.2
7 (12)	97.2 / 95.4	96.8 / 95.0	97.6 / 95.6	97.1 / 95.1	97.4 / 95.4	97.5 / 95.5	98.5 / 96.5

Las desviaciones entre semillas son de 0.03 a 0.8 pp (el árbol: 0.04–0.05 con N = 5 y 7). Con K = 3 el orden es el mismo (con N = 5: control 93.9 / 92.6; árbol 96.95 / 95.5; DDPG 96.8 / 95.4).

Lectura. (1) Con N = 5 y 10 mediciones, DDPG y el árbol ganan ≈ 3 pp de exactitud y de energía a ambos controles. (2) Con N = 7 y 12 mediciones el árbol supera incluso al control con 15 mediciones (98.5 / 96.5 contra 98.4 / 95.9). (3) El árbol es el más barato (2 s de entrenamiento). (4) PPO y REINFORCE quedan algo por debajo de SAC, DDPG y árbol, aunque por encima de los controles con N de 3 a 5. (5) Corrijo una afirmación de la V4: la «cota supervisada» (98.1 %) no era un techo; el árbol, que solo usa potencias medidas, la supera (98.5 %).

Resultados en los 4 casos reales (N = 5, K = 5, final / energía). control 95.5 / 94.2; REINFORCE 97.7 / 95.8; SAC 98.7 / 97.0; DDPG 97.0 / 95.4; PPO 95.9 / 94.0; árbol 94.5 / 92.8. Con N = 7: control 93.5 / 91.7; REINFORCE 96.8 / 94.7; SAC 97.5 / 95.3; DDPG 98.5 / 96.3; árbol 97.7 / 95.5; PPO 94.0 / 91.7. Son solo 4 casos: ver el Paso 6.

Archivos. `algoritmos_V5.py`, `comparar_algoritmos_V5.py`, `pvsim_V5/bandit_env_V5.py`, `resumen_comparar_V5.py`, `resultados_V5/comparar_N*.json`, `resultados_V5/resumen_comparar_V5.txt`.

PASO 5

Paso 5 — Ruido de medición (1 y 2 W)

Qué hice. El mismo experimento del Paso 4 (N = 5 y 7, K = 5, 3 semillas) con ruido gaussiano aditivo de 1 y 2 W en TODAS las potencias leídas (barrido y muestras), tanto en el entrenamiento como en la evaluación.

Resultado (100 sintéticos nuevos, final / energía).

RUIDO, N	CONTROL SIMPLE	REINFORCE	PPO	SAC	DDPG	ÁRBOL
1 W, N = 5	94.6 / 93.2	97.0 / 95.3	96.6 / 95.0	97.1 / 95.5	97.5 / 95.8	97.2 / 95.7
1 W, N = 7	96.9 / 95.1	97.5 / 95.4	97.1 / 95.1	97.5 / 95.5	97.7 / 95.7	98.1 / 96.1
2 W, N = 5	94.5 / 93.1	97.0 / 95.3	96.5 / 94.9	97.0 / 95.4	97.2 / 95.5	96.7 / 95.1
2 W, N = 7	96.7 / 94.9	97.1 / 95.1	97.2 / 95.2	97.3 / 95.3	97.3 / 95.4	97.8 / 95.9

Lectura. (1) La ventaja sobre los controles se mantiene (+2 a +3 pp con N = 5; +0.5 a +1.4 con N = 7). (2) El orden entre métodos cambia un poco: el árbol es el mejor con N = 7, pero con 2 W y N = 5 baja a 96.7 y DDPG pasa al frente; SAC y DDPG parecen más robustos. (3) En los 4 reales, REINFORCE es consistentemente bueno (96.6–97.2) y PPO falla con N = 7 (≈ 93 %).

Alcance. Es ruido estático en el nivel 1; en el nivel dinámico el ruido interactúa con el umbral de 5 W de reinicio, que no se probó.

Archivos. `resultados_V5/comparar_N*_ruido*.json`, `resultados_V5/resumen_comparar_V5.txt`.

PASO 6

Paso 6 — Revisión de los casos reales (¿por qué el árbol falla en algunos?)

Qué hice. `revision_reales_V5.py` : diagnóstico por caso real y prueba de una causa probable, el DESAJUSTE DE DISTRIBUCIÓN. El caso 3 tiene Isc = 3.25 A y, como el rango se fija con la Isc NOMINAL (8.9 A), solo 2 de los 5 o 7 puntos del barrido tienen potencia; en el banco sintético original solo el 2.1 % de los patrones tenían Isc menor a 4.5 A. Entrené el árbol con un banco con ATENUACIÓN GLOBAL (con probabilidad 0.5 todas las irradiancias se multiplican por U[0.25, 1]; el 23 % de los patrones pasan a tener Isc baja), `build_pool_aug` en `reinforce_nn_V5.py` , y lo evalué también en 100 sintéticos NUEVOS con atenuación global (semilla distinta).

Resultado (eficiencia final, K = 5, árbol con 3 semillas).

	REALES (MEDIA DE 4)	SINTÉTICOS ORIGINALES	SINTÉTICOS ATENUADOS
N = 5, control simple	95.6	94.7	91.5
N = 5, árbol (banco original)	94.4	97.7	95.1
N = 5, árbol (con atenuación)	96.0	97.5	96.2
N = 7, control simple	95.0	97.2	95.2
N = 7, árbol (banco original)	96.8	98.5	96.7
N = 7, árbol (con atenuación)	97.0	98.6	97.2

Por caso (N = 5, control / árbol original / árbol con atenuación): caso 1: 99.9 / 99.9 / 99.9; caso 2: 94.9 / 92.5 / 96.3; caso 3: 89.4 / 87.4 / 89.3; caso 4: 98.2 / 97.6 / 98.5.

Lectura. La atenuación global corrige en parte la caída (N = 5: de 94.4 a 96.0 en reales, por encima del control) y mejora los sintéticos atenuados (95.1 → 96.2). El caso 3 sigue siendo el más débil para todos (≈ 87–92 %): con el rango fijado por la Isc nominal casi no hay puntos útiles en el barrido.

Advertencia metodológica. La idea de la atenuación se propuso DESPUÉS de ver el fallo en los reales, por lo que esos 4 casos dejan de ser un conjunto de prueba puro; por eso se validó también con sintéticos nuevos. Las cifras de los Pasos 4 y 5 usan el banco original.

Archivos. `revision_reales_V5.py` , `resultados_V5/revision_reales_V5.txt` .

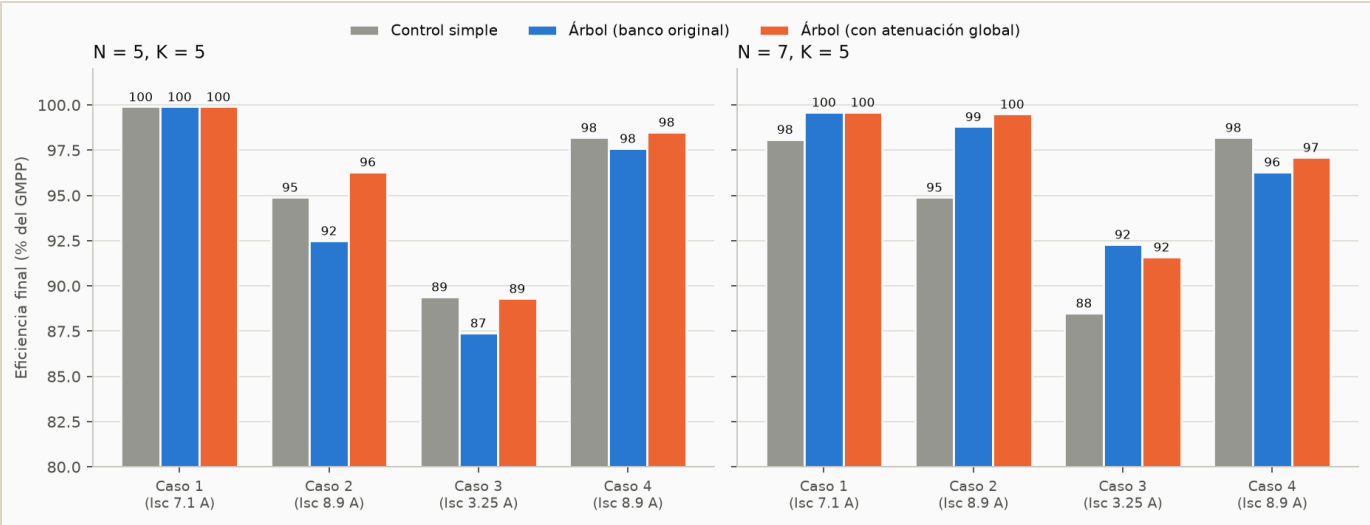


Figura C. Eficiencia final por caso real (K = 5): control simple, árbol con el banco original y árbol entrenado con atenuación global. El caso 3 (Isc = 3.25 A) es el más débil para todos.

3. Conclusiones

Qué se puede afirmar

- El problema de la V4 estaba en el objetivo de entrenamiento y no en la cantidad de entrenamiento: premiar la mejor de las K muestras, mantener σ fija y entrenar con más muestras que en operación permite que los algoritmos aprendan dónde proponer el punto.
- Con pocas mediciones (N de 3 a 7, 6 a 12 mediciones en total), REINFORCE, PPO, SAC, DDPG y el árbol superan a los controles sin aprendizaje, con ventajas de hasta ≈ 3 pp con N = 5 y de ≈ 1.3 pp con N = 7 (árbol); las desviaciones entre semillas son pequeñas.
- El árbol de decisión recalculado en cada iteración, propuesto por el profesor, es el mejor con N = 7 y el más barato de entrenar; con ruido alto y N = 5, DDPG y SAC son más robustos.
- La ventaja persiste con ruido de medición de 1 y 2 W (nivel 1).

Qué no se puede afirmar

- Que supere la exactitud del MSAPSO: el techo con 10–12 mediciones es ≈ 97–98.5 %, contra ≈ 100 % con 175. La ganancia es en costo (≈ 15 veces menos mediciones) y, en la métrica de energía con ventana de 200 mediciones, en energía.
- Que funcione en los casos reales: son solo 4 y los resultados son mixtos; el caso 3 es débil para todos. La corrección con atenuación global se propuso después de ver el fallo.
- Que funcione con la dinámica del convertidor, la sombra gradual o el reinicio por 5 W: solo se probó el nivel 1.

> **Para el profesor.** Fijar el rango con la Isc nominal perjudica a los patrones de Isc baja, tanto del RL como de los controles. Una opción a discutir es reescalar el rango tras la primera medición. También hay que acordar si el criterio de éxito es «menos mediciones con ≈ 2 pp menos de exactitud que el MSAPSO».

4. Siguientes pasos y preguntas para el profesor

Propuesta (no ejecutada en esta entrega):

1. **Nivel dinámico** (Boost + Super-Twisting a 0.5 ms y 5 ms): corregir primero la eficiencia para medirla contra P_ceiling, y comprobar que la ventaja sobreviva a la inercia del inductor.
2. **Reinicio por 5 W** remuestreando los N puntos, y luego aprender el umbral óptimo (el profesor lo dejó para más tarde); incluir la interacción con el ruido.
3. **Sombra que se desplaza gradualmente** (3, 10, 30 y 120 s), aún no implementada.
4. **Rango adaptable a la Isc** tras la primera medición, para los patrones de Isc baja.
5. **Robustez:** más semillas, más patrones reales si los hay, otros σ de evaluación, N de 4, 6, 8 y 10, y optimización bayesiana como referencia sin aprendizaje.

Preguntas para el profesor

1. ¿Acepta la métrica «mediciones» como criterio principal, con ≈ 2 pp menos de exactitud final que el MSAPSO?
2. ¿Reescalamos el rango según la primera medición para no perder los patrones de Isc baja, o mantenemos la Isc nominal fija?
3. La atenuación global en el generador sintético se propuso después de ver los casos reales: ¿lo considera aceptable si se valida con sintéticos nuevos?
4. ¿Qué escenario prefiere priorizar: nivel dinámico, reinicio por 5 W o sombra gradual?

5. Limitaciones y supuestos declarados

- **Nivel 1 solamente** (potencia estática): sin dinámica del convertidor, reinicio por 5 W ni sombra gradual. El ruido es estático, aditivo y gaussiano.
- **Controles diseñados por nosotros:** el control simple y el parabólico; un competidor podría tener uno mejor.
- **Solo 4 casos reales:** sus cifras son frágiles. El diseño de la atenuación global se propuso después de ver su fallo (Paso 6).
- **3 semillas** que varían solo el entrenamiento; el banco de curvas sintéticas de entrenamiento es el mismo (10 000 curvas, semilla 0) y la evaluación usa 100 sintéticos fijos con 3 repeticiones.
- **σ de evaluación fija en 0.05·Isc_nominal;** otros valores pueden cambiar el orden entre métodos. N de 3, 5 y 7 y K de 3 y 5; el ruido solo con N = 5 y 7 y K = 5.
- **Hiperparámetros** por defecto o poco explorados en PPO, SAC y DDPG; el árbol usa profundidad 10 y 15 iteraciones sin búsqueda sistemática.
- **La métrica de energía** (ventana de 200 mediciones) favorece búsquedas cortas; leerla junto a la exactitud final y el número de mediciones.
- **La «cota supervisada»** de la V4 y del Paso 3 usa el GMPP como etiqueta: es un diagnóstico, no un método válido, y no es un techo.
- **Supuestos propios por confirmar:** normalizar por 250 W, σ de evaluación, ventana de 200 mediciones y rango [0.10, 0.95]·Isc_nominal.
- **Dependencia:** el árbol usa scikit-learn.

6. Mapa de archivos y cómo reproducir

Mapa de archivos

ARCHIVO	QUÉ CONTIENE / QUÉ PRODUCE
`diseno_V5.md` , `bitacora_V5.md`	Diseño con las indicaciones del profesor y las correcciones, y registro paso a paso (fuente de este PDF).
`pvsim_V5/`	Simulador (curvas, convertidor, MSAPSO), plantas, métricas y el entorno de un paso `bandit_env_V5.py` .
`datasets_V5/`	Los 4 casos reales del paper.
`baselines_nivel1_V5.py`	Escenarios de evaluación y controles sin aprendizaje (heredado de la V4).
`reinforce_nn_V5.py` , `grid_reinforce_V5.py` → `resultados_V5/reinforce_grid_V5.*`	REINFORCE corregido y su malla (Pasos 1 y 2).
`margen_NK_V5.py` , `control_parabolico_V5.py` → `resultados_V5/margen_NK_V5.txt` , `control_parabolico_V5.txt`	Margen según N y K y control parabólico (Paso 3).
`algoritmos_V5.py` , `comparar_algoritmos_V5.py` , `resumen_comparar_V5.py` → `resultados_V5/comparar_N*.json` , `resumen_comparar_V5.txt`	PPO, SAC, DDPG, árbol y REINFORCE sobre la misma formulación, con y sin ruido (Pasos 4 y 5).
`revision_reales_V5.py` → `resultados_V5/revision_reales_V5.txt`	Diagnóstico de los casos reales y atenuación global (Paso 6).
`graficas_V5.py` , `construir_reporte_V5.py`	Figuras y compilación de este PDF.

Cómo reproducir (con `requirements_V5.txt` , Python 3.10 o superior)

```
python3 baselines_nivel1_V5.py           # controles sin aprendizaje
python3 grid_reinforce_V5.py             # Paso 2: malla de REINFORCE (unos minutos)
python3 margen_NK_V5.py                  # Paso 3: margen segun N y K
python3 control_parabolico_V5.py         # Paso 3: control parabolico
for N in 3 5 7; do python3 comparar_algoritmos_V5.py --N $N --seeds 0 1 2 & done; wait       # Paso 4 (~15 min en paralelo)
for N in 5 7; do for ns in 1.0 2.0; do python3 comparar_algoritmos_V5.py --N $N --noise $ns --seeds 0 1 2 & done; done; wait   #
Paso 5
python3 revision_reales_V5.py            # Paso 6
python3 resumen_comparar_V5.py           # tabla resumen
python3 graficas_V5.py && python3 construir_reporte_V5.py     # figuras y PDF (Chrome en macOS y pdftotext)
```

Todo corre en CPU. Los entrenamientos del nivel 1 toman segundos o pocos minutos (SAC y DDPG, 2–4 minutos por semilla).

ANEXO

Anexo. Código clave

pvsim_V5/bandit_env_V5.py

42 LÍNEAS

```
"""
bandit_env_V5 — Entorno Gymnasium de UN PASO (bandido contextual) para PPO / SAC / DDPG en la formulacion corregida.
Episodio = un patron de sombra: observacion = N potencias medidas / 250 W (barrido ascendente); accion = corriente a probar;
recompensa (reward="gain") = max(0, P_medida(a) - mejor punto del barrido) / 250 W = cuanto MEJORA lo que ya se conocia (solo potencias
medidas;
sin GMPP). reward="abs" = P_medida/250 (la de la V4). Usa un banco precalculado de curvas (tablas P(I)) para que cada episodio sea instantaneo.
El muestreo de K puntos aledanos se hace al EVALUAR (mu + gaussiana) o lo hace la politica (PPO/SAC); el entrenamiento premia que la
accion propuesta supere al mejor punto del barrido.
"""
import numpy as np
import gymnasium as gym
from gymnasium import spaces
from pvsim_V5.medicion_V5 import ISC_NOM, I_MIN, I_MAX, P_NOM, grid_currents, action_to_current

TABLE_PTS = 600
IGRID = np.linspace(0.0, ISC_NOM, TABLE_PTS)


class MPPTBanditEnvV5(gym.Env):
    metadata = {"render_modes": []}

    def __init__(self, N, pool, obs_all, reward="gain", noise_w=0.0, seed=0):
        super().__init__()
        self.N, self.pool, self.obs_all, self.reward_mode, self.noise_w = N, pool, obs_all, reward, noise_w
        self.action_space = spaces.Box(-1.0, 1.0, shape=(1,), dtype=np.float32)
        self.observation_space = spaces.Box(0.0, 10.0, shape=(N,), dtype=np.float32)
        self._rng = np.random.default_rng(seed)

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        if seed is not None:
            self._rng = np.random.default_rng(seed)
        self.i = int(self._rng.integers(0, self.pool.shape[0]))
        o = self.obs_all[self.i] + (self._rng.normal(0, self.noise_w, self.N) / P_NOM if self.noise_w > 0 else 0.0)
        self.obs = np.clip(o, 0.0, 10.0).astype(np.float32)
        return self.obs.copy(), {}

    def step(self, action):
        I = action_to_current(float(np.asarray(action).reshape(-1)[0]))
        P = float(np.interp(I, IGRID, self.pool[self.i], right=0.0)) + (self._rng.normal(0, self.noise_w) if self.noise_w > 0 else 0.0)
        Pn = P / P_NOM
        r = max(0.0, Pn - float(self.obs.max())) if self.reward_mode == "gain" else Pn
        return self.obs.copy(), r, True, False, {}
```

algoritmos_V5.py

102 LÍNEAS

```
"""
algoritmos_V5.py — Entrenamiento y evaluacion de TODOS los algoritmos sobre la MISMA formulacion corregida (nivel 1, sin GMPP en
entrenamiento):
  reinforce : REINFORCE-MLP, recompensa de grupo &#x27;gain', baseline de valor, sigma fija (reinforce_nn_V5)
  ppo       : PPO (sb3), episodio de 1 paso, recompensa &#x27;gain', sigma FIJA (log_std congelada)
  sac       : SAC (sb3), episodio de 1 paso, recompensa &#x27;gain'
  ddpg      : DDPG (sb3), politica determinista + ruido gaussiano de exploracion, recompensa &#x27;gain'
  arbol     : arbol de decision DENTRO del bucle de RL, RECALCULADO en cada iteracion (propuesta del profesor):
              en cada iteracion se muestrean K acciones alrededor de la propuesta actual, se mide la potencia y el arbol se REAJUSTA con
              la mejor accion MEDIDA de cada patron (si mejora al barrido); sigma decrece entre iteraciones. Solo potencias medidas.
Evaluacion comun: se mide el barrido de N puntos, el modelo propone mu y se toman K muestras N(mu, sigma_eval^2); se elige la mejor LEIDA.
"""
import sys, os, time
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
import torch
from pvsim_V5.medicion_V5 import ISC_NOM, I_MIN, I_MAX, P_NOM, grid_currents, scan, metrics_from_plant, action_to_current, current_to_action
from pvsim_V5.static_plant_V5 import StaticPlant
from pvsim_V5.bandit_env_V5 import MPPTBanditEnvV5, IGRID
from reinforce_nn_V5 import build_pool, table_lookup, sigma_action, train as train_reinforce

SIGMA_EVAL = 0.05
```

```
def make_obs_all(pool, N):
    """Potencias del barrido ascendente (N corrientes uniformes) para cada curva del banco, / 250 W."""
    return np.stack([table_lookup(pool, np.tile(i, (pool.shape[0], 1)))[:, 0] for i in grid_currents(N)], axis=1) / P_NOM


def eval_mu(mu_fn, N, K, sigma, curves, mode="gauss", reps=1, noise_w=0.0, seed=11):
    """mu_fn(obs_vector) -> accion en [-1,1]. mode &#x27;gauss': N + K muestras N(mu, sigma); 'mu': N + 1 en mu."""
    sa = sigma_action(sigma); out = []
    for k, c in enumerate(curves):
        for r in range(reps):
            rng = np.random.default_rng(seed + 1000 * k + r)
            pl = StaticPlant(c, noise_w=noise_w, rng=rng)
            _, P = scan(pl, N)
            mu = float(np.clip(mu_fn(np.clip(P / P_NOM, 0, 10).astype(np.float32)), -1, 1))
            if mode == "mu":
                pl.run(1.0, action_to_current(mu))
            else:
                for _ in range(K):
                    pl.run(1.0, action_to_current(mu + sa * rng.normal()))
            out.append(metrics_from_plant(pl, c))
    return {m: float(np.nanmean([o[m] for o in out])) for m in out[0]}


# ------------------------------------------------------------------ arbol dentro del RL
class TreeMu:
    def __init__(self, tree, init_fn):
        self.tree, self.init_fn = tree, init_fn

    def __call__(self, obs):
        return float(self.tree.predict(obs[None])[0]) if self.tree is not None else float(self.init_fn(obs[None])[0])


def _init_mu(obs):                                  # propuesta inicial: la corriente del mejor punto del barrido (potencia MEDIDA)
    N = obs.shape[1]; return current_to_action(grid_currents(N)[np.argmax(obs, axis=1)])


def train_tree(N, pool, obs_all, iters=15, B=4000, K=10, sigma0=0.15, sigma1=0.04, depth=10, min_leaf=8, seed=0, noise_w=0.0):
    from sklearn.tree import DecisionTreeRegressor
    rng = np.random.default_rng(seed); tree = None
    for t in range(iters):
        sig = sigma0 + (sigma1 - sigma0) * t / max(iters - 1, 1); sa = sigma_action(sig)
        idx = rng.integers(0, pool.shape[0], B); obs = obs_all[idx] + (rng.normal(0, noise_w, (B, obs_all.shape[1])) / P_NOM if noise_w > 0
else 0.0)
        mu = _init_mu(obs) if tree is None else np.clip(tree.predict(obs), -1, 1)
        a = np.clip(mu[:, None] + sa * rng.normal(size=(B, K)), -1, 1)
        I = I_MIN + 0.5 * (a + 1.0) * (I_MAX - I_MIN)
        Pn = table_lookup(pool[idx], I) / P_NOM + (rng.normal(0, noise_w, (B, K)) / P_NOM if noise_w > 0 else 0.0)
        j = np.argmax(Pn, axis=1); best_a = a[np.arange(B), j]; improved = Pn.max(axis=1) > obs.max(axis=1)
        target = np.where(improved, best_a, mu)          # solo se mueve el objetivo si la muestra MEJORA al barrido (potencia medida)
        tree = DecisionTreeRegressor(max_depth=depth, min_samples_leaf=min_leaf, random_state=seed).fit(obs, target)   # se RECALCULA en cada
iteracion
    return TreeMu(tree, _init_mu)


# ------------------------------------------------------------------ algoritmos de stable-baselines3
def train_sb3(algo, N, pool, obs_all, steps, seed, noise_w=0.0):
    from stable_baselines3 import PPO, SAC, DDPG
    from stable_baselines3.common.vec_env import DummyVecEnv
    from stable_baselines3.common.noise import NormalActionNoise
    sa = sigma_action(SIGMA_EVAL)
    if algo == "ppo":
        venv = DummyVecEnv([lambda i=i: MPPTBanditEnvV5(N, pool, obs_all, "gain", noise_w, seed=seed * 100 + i) for i in range(8)])
        m = PPO("MlpPolicy", venv, seed=seed, gamma=0.0, gae_lambda=0.0, n_steps=256, batch_size=256, policy_kwargs=dict(net_arch=[64, 64]),
verbose=0)
        m.policy.log_std.data.fill_(float(np.log(sa))); m.policy.log_std.requires_grad_(False)      # sigma FIJA
    else:
        env = DummyVecEnv([lambda: MPPTBanditEnvV5(N, pool, obs_all, "gain", noise_w, seed=seed)])
        kw = dict(policy_kwargs=dict(net_arch=[64, 64]), learning_starts=2000, seed=seed, verbose=0, gamma=0.0)
        if algo == "sac":
            m = SAC("MlpPolicy", env, **kw)
        else:
            m = DDPG("MlpPolicy", env, action_noise=NormalActionNoise(np.zeros(1), sa * np.ones(1)), **kw)
    m.learn(steps)
    return m


def sb3_mu(model):
    return lambda obs: float(model.predict(obs, deterministic=True)[0][0])


def reinforce_mu(pol):
    return lambda obs: float(pol(torch.tensor(obs, dtype=torch.float32)[None])[0])
```

reinforce_nn_V5.py 137 LÍNEAS

```
"""
reinforce_nn_V5.py — REINFORCE con MLP, FORMULACION CORREGIDA (V5). Nivel 1 (potencia estatica), sin GMPP en entrenamiento.

Correcciones respecto a la V4 (cada una ataca la causa probable del resultado de la V4: se optimizaba la potencia PROMEDIO de las
muestras, lo que empuja sigma a 0 y no premia al MEJOR punto, que es el que se elige al final):
  (a) reward = "max":  recompensa de GRUPO = max_k P_k (la mejor muestra de los K, potencia MEDIDA) -> objetivo J = E[max_k P(a_k)].
      reward = "gain": recompensa de GRUPO = max(0, max_k P_k - mejor punto del barrido), solo cuenta cuanto MEJORA lo ya conocido.
      reward = "mean": la de la V4 (potencia de cada muestra), para comparar.
      Gradiente de grupo:  grad J = E[ (G - b) * sum_k (a_k - mu)/sigma^2 * grad mu ]
  (b) sigma FIJA (sigma1 = None) o DECRECIENTE (de sigma0 a sigma1, lineal en las actualizaciones): no se aprende sigma, no puede colapsar.
  (c) K de entrenamiento (K_train) distinto del K de evaluacion: p. ej. 20 muestras al entrenar y 5 al evaluar.
Baselines b: "none" | "batch" (media movil de G) | "value" (V(obs) aprendida).
USO: python3 reinforce_nn_V5.py --reward max --baseline value --K-train 20 --sigma0 0.05
"""
import sys, os, json, time, argparse
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
import torch, torch.nn as nn
from pvsim_V5.array_model_V5 import SyntheticPVCurve, random_shading_mix
from pvsim_V5.static_plant_V5 import StaticPlant
from pvsim_V5.medicion_V5 import (ISC_NOM, I_MIN, I_MAX, P_NOM, grid_currents, scan, metrics_from_plant, action_to_current)

TABLE_PTS = 600


def p_table(curve):
    Ig = np.linspace(0.0, ISC_NOM, TABLE_PTS)
    return np.interp(Ig, curve.I, curve.P, right=0.0).astype(np.float32)


def build_pool(M, seed=0):
    rng = np.random.default_rng(seed)
    return np.stack([p_table(SyntheticPVCurve(random_shading_mix(rng))) for _ in range(M)])


def table_lookup(tab, I):
    pos = np.clip(I / ISC_NOM, 0.0, 1.0) * (TABLE_PTS - 1)
    lo = np.floor(pos).astype(int); hi = np.minimum(lo + 1, TABLE_PTS - 1); fr = pos - lo
    rows = np.arange(tab.shape[0])[:, None]
    return tab[rows, lo] * (1 - fr) + tab[rows, hi] * fr


class Policy(nn.Module):
    def __init__(self, N, hidden=64):
        super().__init__()
        self.mu = nn.Sequential(nn.Linear(N, hidden), nn.ReLU(), nn.Linear(hidden, hidden), nn.ReLU(), nn.Linear(hidden, 1), nn.Tanh())
        self.v = nn.Sequential(nn.Linear(N, hidden), nn.ReLU(), nn.Linear(hidden, 1))

    def forward(self, x):
        return self.mu(x).squeeze(-1)


def sigma_action(sigma_frac):
    return sigma_frac * ISC_NOM / (0.5 * (I_MAX - I_MIN))


def train(N=7, K_train=20, sigma0=0.05, sigma1=None, baseline="value", reward="max", updates=4000, B=128, lr=1e-3,
          noise_w=0.0, seed=0, pool=None, log=False):
    torch.manual_seed(seed); rng = np.random.default_rng(seed)
    pool = pool if pool is not None else build_pool(6000, seed=0)
    Iscan = grid_currents(N)
    obs_all = np.stack([table_lookup(pool, np.tile(i, (pool.shape[0], 1)))[:, 0] for i in Iscan], axis=1) / P_NOM
    pol = Policy(N); opt = torch.optim.Adam(pol.parameters(), lr=lr)
    b_run, t0, K = 0.0, time.time(), K_train
    for u in range(updates):
        frac = u / max(updates - 1, 1)
        sig = sigma0 if sigma1 is None else sigma0 + (sigma1 - sigma0) * frac
        sa = sigma_action(sig)
        idx = rng.integers(0, pool.shape[0], B)
        obs = obs_all[idx] + (rng.normal(0, noise_w, (B, N)) / P_NOM if noise_w > 0 else 0.0)
        x = torch.tensor(obs, dtype=torch.float32)
        mu = pol(x)
        a = mu[:, None].detach() + sa * torch.randn(B, K)
        I = I_MIN + 0.5 * (np.clip(a.numpy(), -1, 1) + 1.0) * (I_MAX - I_MIN)
        P = table_lookup(pool[idx], I) + (rng.normal(0, noise_w, (B, K)) if noise_w > 0 else 0.0)
        Pn = P / P_NOM                                                  # SOLO potencia medida
        scan_best = obs.max(axis=1)
        if reward == "mean":                                            # V4: recompensa por muestra
            G = torch.tensor(Pn, dtype=torch.float32)                   # (B,K)
        elif reward == "max":
            G = torch.tensor(Pn.max(axis=1), dtype=torch.float32)       # (B,)
        elif reward == "gain":
            G = torch.tensor(np.maximum(Pn.max(axis=1) - scan_best, 0.0), dtype=torch.float32)
        else:
            raise ValueError(reward)
        if baseline == "value":
            v = pol.v(x).squeeze(-1)
            b = v.detach()[:, None] if reward == "mean" else v.detach()
        elif baseline == "batch":
```

```
            b_run = 0.95 * b_run + 0.05 * float(G.mean()); b = b_run
        else:
            b = 0.0
        adv = G - b
        adv = adv / (adv.std() + 1e-6) if baseline != "none" else adv
        score = (a - mu.detach()[:, None]) / sa ** 2                        # (B,K)
        if reward == "mean":
            loss = -(adv * score * mu[:, None]).mean()
        else:
            loss = -(adv * score.mean(dim=1) * mu).mean()                   # score de grupo (promedio de las K)
        if baseline == "value":
            tgt = G.mean(dim=1) if reward == "mean" else G
            loss = loss + ((pol.v(x).squeeze(-1) - tgt) ** 2).mean()
        opt.zero_grad(); loss.backward(); opt.step()
        if log and (u % 1000 == 0 or u == updates - 1):
            print(f"  upd {u:5d}  G medio {float(G.mean()):.4f}  sigma {sig:.3f}  ({time.time()-t0:.0f} s)", flush=True)
    return pol


def eval_policy(pol, N, K, sigma, curves, mode="stoch", noise_w=0.0, seed=11, reps=1):
    """mode &#x27;stoch': N + K muestras gaussianas.  'mu': N + 1 en mu.  'mu+stoch': N + 1 + K."""
    sa = sigma_action(sigma); out = []
    for k, c in enumerate(curves):
        for r in range(reps):
            rng = np.random.default_rng(seed + 1000 * k + r)
            pl = StaticPlant(c, noise_w=noise_w, rng=rng)
            _, P = scan(pl, N)
            with torch.no_grad():
                mu = float(pol(torch.tensor(P / P_NOM, dtype=torch.float32)[None])[0])
            if mode in ("mu", "mu+stoch"):
                pl.run(1.0, action_to_current(mu))
            if mode in ("stoch", "mu+stoch"):
                for _ in range(K):
                    pl.run(1.0, action_to_current(mu + sa * rng.normal()))
            out.append(metrics_from_plant(pl, c))
    return {m: float(np.nanmean([o[m] for o in out])) for m in out[0]}


def build_pool_aug(M, seed=0, p_global=0.5, smin=0.25):
    """Banco con ATENUACION GLOBAL: con prob. p_global todas las irradiancias se multiplican por U[smin,1] (cielo nublado / suciedad general),
    lo que produce patrones de Isc baja que el generador original casi no produce. Solo curvas sinteticas."""
    rng = np.random.default_rng(seed); tabs = []
    for _ in range(M):
        irr = random_shading_mix(rng)
        if rng.random() < p_global:
            irr = irr * rng.uniform(smin, 1.0)
        tabs.append(p_table(SyntheticPVCurve(irr)))
    return np.stack(tabs)
```

comparar_algoritmos_V5.py

35 LÍNEAS

```
"""
comparar_algoritmos_V5.py — Entrena y evalua REINFORCE, PPO, SAC, DDPG y arbol (formulacion corregida) para un N dado, varias semillas.
Evalua en 100 sinteticos nuevos (reps=3) y 4 reales (reps=5), con K = 3 y 5 muestras, sigma_eval = 0.05, y tambien solo en mu.
USO: python3 comparar_algoritmos_V5.py --N 5 --seeds 0 1 2 [--steps-onpolicy 300000 --steps-offpolicy 60000]
"""
import sys, os, json, time, argparse
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np, torch
torch.set_num_threads(1)
from algoritmos_V5 import *
from baselines_nivel1_V5 import scenarios

ap = argparse.ArgumentParser()
ap.add_argument("--N", type=int, required=True); ap.add_argument("--seeds", type=int, nargs="+", default=[0, 1, 2])
ap.add_argument("--steps-onpolicy", type=int, default=300000); ap.add_argument("--steps-offpolicy", type=int, default=60000)
ap.add_argument("--noise", type=float, default=0.0); ap.add_argument("--algos", nargs="+", default=["reinforce", "ppo", "sac", "ddpg",
"arbol"])
a = ap.parse_args()
real, synth = scenarios(); pool = build_pool(10000, seed=0); obs_all = make_obs_all(pool, a.N)
res = {}
for seed in a.seeds:
    for algo in a.algos:
        t0 = time.time()
        if algo == "reinforce":
            mus = {K: reinforce_mu(train_reinforce(a.N, K, SIGMA_EVAL, None, "value", "gain", 4000, seed=seed, pool=pool, noise_w=a.noise)) for
K in (3, 5)}
        elif algo == "arbol":
            m = train_tree(a.N, pool, obs_all, seed=seed, noise_w=a.noise); mus = {3: m, 5: m}
        else:
            m = sb3_mu(train_sb3(algo, a.N, pool, obs_all, a.steps_onpolicy if algo == "ppo" else a.steps_offpolicy, seed, a.noise)); mus = {3:
m, 5: m}
        for K in (3, 5):
            for mode in ("gauss", "mu"):
                s = eval_mu(mus[K], a.N, K, SIGMA_EVAL, synth, mode, reps=3, noise_w=a.noise)
```

```
                r = eval_mu(mus[K], a.N, K, SIGMA_EVAL, real, mode, reps=5, noise_w=a.noise)
                res.setdefault(f"{algo}|K{K}|{mode}", []).append([s["eff_final"], s["eff_energy"], r["eff_final"], r["eff_energy"]])
        print(f"N={a.N} seed={seed} {algo:9s} listo en {time.time()-t0:.0f} s | K=5 gauss: sint {res[f&#x27;{algo}|K5|gauss'][-1][0]:.2f}/{res[f'{algo}|K5|gauss'][-1][1]:.2f}", flush=True)
        json.dump(res, open(f"resultados_V5/comparar_N{a.N}{&#x27;_ruido'+str(a.noise) if a.noise else ''}.json", "w"), indent=1)
```

control_parabolico_V5.py

36 LÍNEAS

```
"""Control sin aprendizaje MAS FUERTE: del barrido de N puntos se ajusta una parabola a los 3 puntos vecinos del mejor (interpolacion
parabolica) y se toma su vertice como centro; luego K muestras gaussianas alrededor (y el mejor medido). Es lo que haria un competidor sin RL.
Se compara con &#x27;mejor de N + K gauss.' (control simple) y con el RL corregido (reward gain, baseline value, sigma 0.05, K_train = K)."""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
import numpy as np
from pvsim_V5.static_plant_V5 import StaticPlant
from pvsim_V5.medicion_V5 import *
from baselines_nivel1_V5 import scenarios, make_best_of_n, evaluate
from reinforce_nn_V5 import train, eval_policy, build_pool


def make_parabolic(N, K, sigma_frac=0.05):
    def f(plant, curve, rng):
        I, P = scan(plant, N); j = int(np.argmax(P)); mu = I[j]
        if 0 < j < N - 1:
            y0, y1, y2 = P[j - 1], P[j], P[j + 1]; den = y0 - 2 * y1 + y2
            if den < -1e-9:
                mu = I[j] + 0.5 * (I[1] - I[0]) * (y0 - y2) / den
        mu = float(np.clip(mu, I_MIN, I_MAX))
        for _ in range(K):
            plant.run(1.0, float(np.clip(rng.normal(mu, sigma_frac * ISC_NOM), I_MIN, I_MAX)))
    return f

if __name__ == "__main__":
    real, synth = scenarios(); pool = build_pool(6000, seed=0)
    print("N  K  med | simple fin  ener | parabolico fin  ener | RL corregido fin  ener   [100 sinteticos | (reales: parabolico / RL)]")
    for N, K in ((3, 3), (4, 3), (5, 3), (5, 5), (7, 3), (7, 5), (10, 5)):
        a = evaluate(make_best_of_n(N, K, 0.05), synth); b = evaluate(make_parabolic(N, K), synth); br = evaluate(make_parabolic(N, K), real)
        rl, rr = [], []
        for seed in range(3):
            pol = train(N, K, 0.05, None, "value", "gain", 4000, seed=seed, pool=pool)
            s = eval_policy(pol, N, K, 0.05, synth, "stoch", reps=3); r = eval_policy(pol, N, K, 0.05, real, "stoch", reps=5)
            rl.append((s["eff_final"], s["eff_energy"])); rr.append((r["eff_final"], r["eff_energy"]))
        rl, rr = np.mean(rl, 0), np.mean(rr, 0)
        print(f"{N:2d} {K:2d} {N+K:3d} | {a[&#x27;eff_final']:10.2f} {a['eff_energy']:5.2f} | {b['eff_final']:13.2f} {b['eff_energy']:5.2f} | {rl[0]:12.2f} {rl[1]:5.2f}   | reales: parab {br['eff_final']:.1f}/{br['eff_energy']:.1f}  RL {rr[0]:.1f}/{rr[1]:.1f}", flush=True)
```